

BCSE304L

Theory of Computation

Dr. ISLABUDEEN M

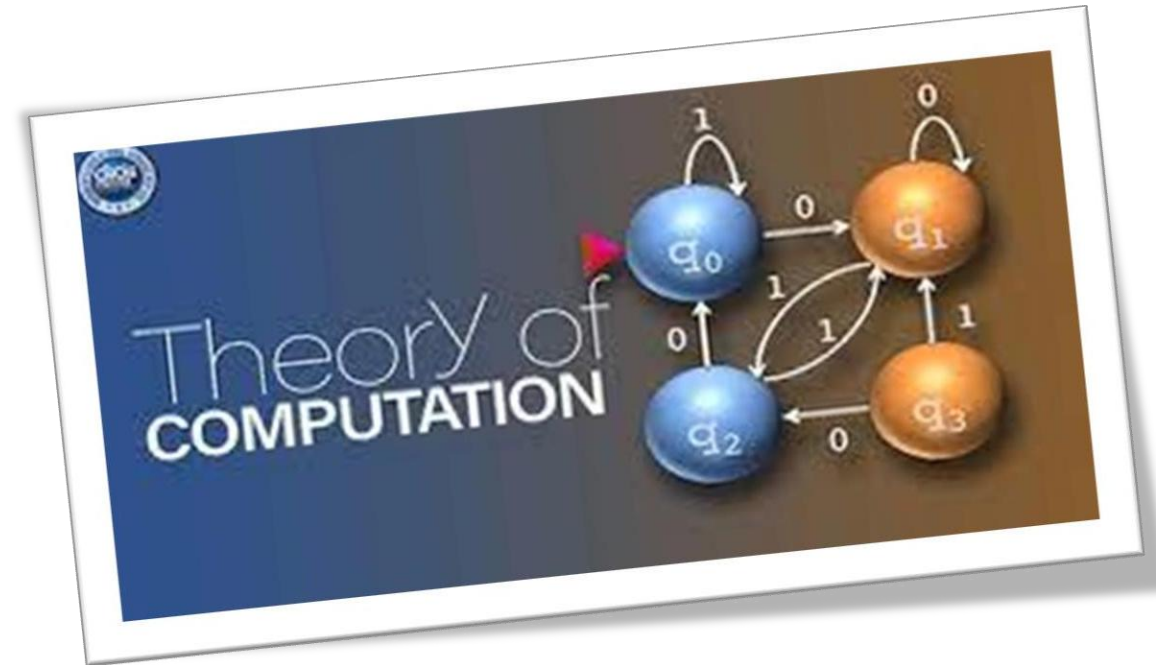
Assistant Professor Senior Grade 2

School of Computer Science and Engineering,
Vellore Institute of Technology,
Vellore, Tamil Nadu, India.

Phone No: +91 9994595561

Mail ID: islabudeen.m@vit.ac.in

Location: PRP 315 AB 23



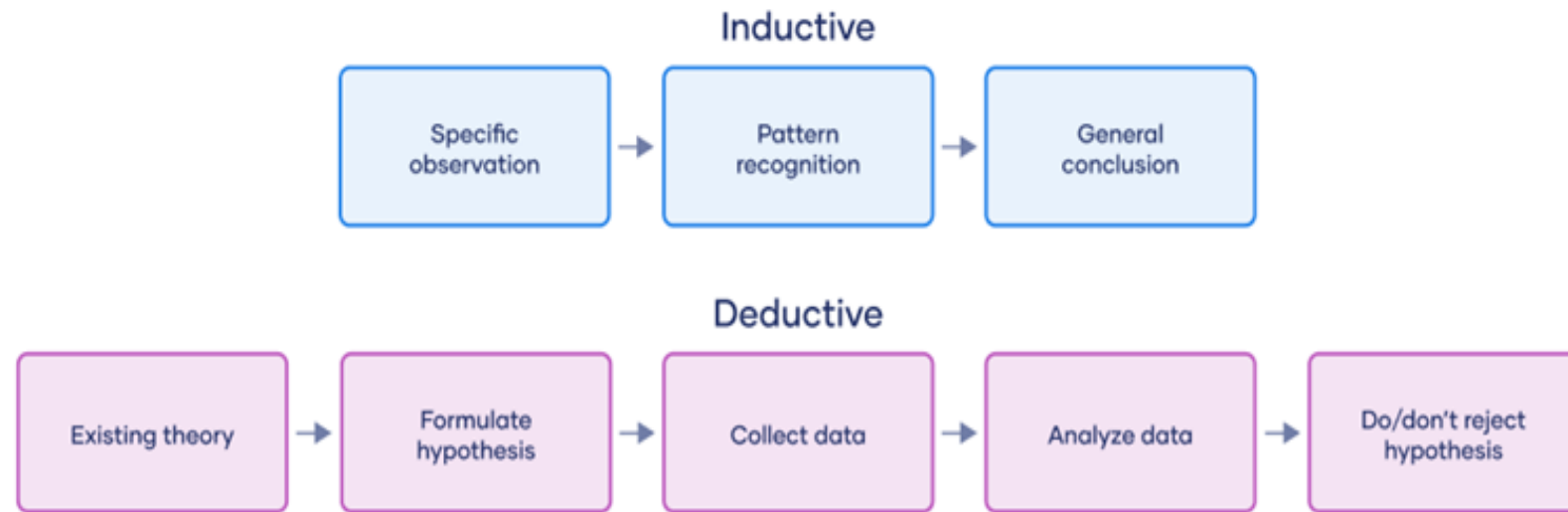
Introduction to Formal Proof

- ❑ **Formal Proof** in the context of the **Theory of Computation** involves using **deductive** and **inductive methods** to establish the **truth of statements**.
- ❑ **Deductive and inductive methods** are two fundamental approaches to reasoning and research, with deductive reasoning testing existing theories and inductive reasoning developing new theories based on observations.
- ❑ The goal of formal proofs is to demonstrate that a conclusion B follows from a hypothesis A, expressed as "**if A then B.**"
- ❑ In TOC, formal proofs are essential for proving theorems and establishing the correctness of algorithms.
- ❑ The study of formal proofs is crucial for understanding the capabilities and limitations of computation and for developing efficient algorithms that can solve complex problems.

Introduction to Formal Proof

- ❖ The main difference between inductive and deductive reasoning is that inductive reasoning aims at **developing a theory** while deductive reasoning aims at **testing an existing theory**.

Inductive vs. deductive reasoning



Deductive Proofs

- Consists of a sequence of statements whose truth lead us from some initial statement called the **hypothesis** / given statement to a **conclusion** statement
- **Hypothesis may be true / false**, typically depending on the values of its parameters
- When we go from **hypothesis H** to **conclusion C**
 - IF **H** THEN **C** $\rightarrow$ **C** is deduced from **H**
- Ex:
 - If **$x \geq 4$** , then **$2^x \geq x^2$**
 - If **x is the sum of squares of four positive integers**, then **$2^x \geq x^2$**

Additional Forms of Proof

1. Proving Equivalence about sets
2. Proofs by contradiction
3. Proofs by counter example

1. Proving equivalence about sets

- If E and F are two expressions representing sets, the statement $E = F$ means that the two sets represented are the same.
- We can **prove the equivalence of Sets $E = F$** in the following ways:
 1. By **commutative law of union of two sets** R and S as $R \cup S = S \cup R$
Where E is the expression for $R \cup S$ and F is the expression for $S \cup R$
So, $E = F$ by **commutative law of union**

Cont..

2. We can prove the set equality $E = F$ by **if and only if statement**, that is “**an element 'x' is in E iff 'x' is in F**”
3. The set equality $E = F$ can be proved by two if statements as follows:
 - i. **If 'x' is in E, then 'x' is in F**
 - ii. **If 'x' is in F, then 'x' is in E**

Contrapositive

- The contrapositive of the statement “if H then C” is “if not C then not H”, and it is represented as

$$H \rightarrow C \Leftrightarrow \neg C \rightarrow \neg H$$

- To prove that “if H then C” and “if not C, then not H” are logically equivalent, there are four cases to consider:
 - H and C both are True
 - H is True and C is False
 - C is True and H is False
 - H and C both are False

Example

- “If ‘x’ is in E then ‘x’ is in F” and the contrapositive statement is

“if ‘x’ is not in F then ‘x’ is not in E”

2. Proof by contradiction

- Another way to prove the statement “if H then C” by the form,

H and not C implies contradiction

- Proof by Contradiction:

- We assume that the Hypothesis and the negation of the conclusion as true and then we completed the proof by showing that something known to be false.

Example

- Prove the following statement by contradiction:

The sum of two even numbers is always even.

- Soln:

- Negate original statement:

The sum of two even numbers is not always even.

Take two even numbers, say 2 and 4, sum them ($2 + 4 = 6$) which is an even.

Take two even numbers, say 12 and 16, sum them ($12 + 16 = 28$) which is an even.

Therefore, by **contradiction**, the sum of two even numbers is always even

3. Counter Examples

- To prove theorem itself is true or false
- To solve this problem, we try to prove the theorem else if we are unable to prove the theorem, then we try to prove that the theorem statement is False.
- It is easy to prove that a statement is not a theorem than to prove it is a theorem.

Example:

- Theorem: All primes are odd

Proof: if x is prime, then x is odd

Disproof: if 2 is prime, then 2 is even [Not odd]

- Theorem: There is no pairs of integers 'a' and 'b' such that " $a \bmod b = b \bmod a$ "

Proof: If a and b are integers, then $a \bmod b \neq b \bmod a$

Disproof: Let $a = b = 2$, then $a \bmod b = b \bmod a \rightarrow 2 \bmod 2 = 2 \bmod 2 = 0$

Inductive Proofs

- Deals with **recursively defined objects** like trees and expressions of various kinds such as regular expression
- Inductive proof includes the following:
 - a. Induction on Integers
 - b. Structural Induction
 - c. Mutual Induction

a. Induction on Integers

- Suppose if we are given a statement, $S(n)$ about an integer 'n' which is to be proved, then we have to prove the following:
 - **Basis:** We prove $S(n)$ for particular integer 'n' when $n = 0, 1, 2$
 - **Induction:** If $S(n)$ then $S(n+1)$, when $n \geq i$, where 'i' is basis integer
- General form of induction: $S(i) S(i+1) S(i+2) \dots S(n)$
- **Induction Principle:**
 - If we prove $S(i)$ and we prove that for all $n \geq i$, $S(n)$ implies $S(n+1)$ then we conclude $S(n)$ for all $n \geq i$

Proof by mathematical induction

Proof by mathematical induction consists of three basic steps. If the statement **S** is to be proved then:

1) Show that **S** is true for some particular integer n_0

- this is called **Basis**

2) Assume **S** is true for some particular integer $k \geq n_0$

-this is called **Induction hypothesis**

3) Then to prove **S** is true for $k+1$

- this is called **Induction step**

Problem 2

Prove the following by induction for all $n \geq 1$,
 $1^2 + 2^2 + \dots + n^2 = n(n+1)(2n+1)/6$

Proof:

For $n = 1$, the statement reduces to $1^2 = \frac{1 \cdot 2 \cdot 3}{6}$ and is obviously true.

Assuming the statement is true for $n = k$:

$$1^2 + 2^2 + 3^2 + \dots + k^2 = \frac{k(k+1)(2k+1)}{6}, \quad (1)$$

we will prove that the statement must be true for $n = k + 1$:

$$1^2 + 2^2 + 3^2 + \dots + (k+1)^2 = \frac{(k+1)(k+2)(2k+3)}{6}. \quad (2)$$

The left-hand side of (2) can be written as

$$1^2 + 2^2 + 3^2 + \dots + k^2 + (k+1)^2.$$

In view of (1), this simplifies to:

$$\begin{aligned} (1^2 + 2^2 + 3^2 + \dots + k^2) + (k+1)^2 &= \frac{k(k+1)(2k+1)}{6} + (k+1)^2 \\ &= \frac{k(k+1)(2k+1) + 6(k+1)^2}{6} \\ &= \frac{(k+1)[k(2k+1) + 6(k+1)]}{6} \\ &= \frac{(k+1)(2k^2 + 7k + 6)}{6} \\ &= \frac{(k+1)(k+2)(2k+3)}{6}. \end{aligned}$$

Problem 3

Prove $1^3 + 2^3 + \dots + n^3 = n^2(n+1)^2/4$ by mathematical induction for $n \geq 0$.

Proof:

For $n = 1$, the statement reduces to $1^3 = \frac{1^2 \cdot 2^2}{4}$ and is obviously true.

Assuming the statement is true for $n = k$:

$$1^3 + 2^3 + 3^3 + \dots + k^3 = \frac{k^2(k+1)^2}{4}, \quad (5)$$

we will prove that the statement must be true for $n = k + 1$:

$$1^3 + 2^3 + 3^3 + \dots + (k+1)^3 = \frac{(k+1)^2(k+2)^2}{4}. \quad (6)$$

The left-hand side of (6) can be written as

$$1^3 + 2^3 + 3^3 + \dots + k^3 + (k+1)^3.$$

In view of (5), this simplifies to:

$$\begin{aligned} (1^3 + 2^3 + 3^3 + \dots + k^3) + (k+1)^3 &= \frac{k^2(k+1)^2}{4} + (k+1)^3 \\ &= \frac{k^2(k+1)^2 + 4(k+1)^3}{4} \\ &= \frac{(k+1)^2[k^2 + 4(k+1)]}{4} \\ &= \frac{(k+1)^2(k+2)^2}{4}. \end{aligned}$$

Problem 4

Prove that $S(n) = a^n - b^n$ is divisible by $(a-b)$ for all $n > 0$

Let $P(n)$ denote the statement $a^n - b^n$ is divisible by $a - b$.

Put $n = 1$. Then $P(1)$ is the statement: $a^1 - b^1 = a - b$ is divisible by $a - b$

$\therefore P(1)$ is true. Now assume that the statement be true for $n = k$

(i.e.,) assume $P(k)$ be true, (i.e.,) $a^k - b^k$ is divisible by $(a - b)$ be true.

$$\Rightarrow \frac{a^k - b^k}{a - b} = m \text{ (say) where } m \in \mathbb{N}$$

$$\Rightarrow a^k - b^k = m(a - b)$$

$$\Rightarrow a^k = b^k + m(a - b) \dots\dots (1)$$

Now to prove $P(k + 1)$ is true, (i.e.,) to prove: $a^{k+1} - b^{k+1}$ is divisible by $a - b$

Consider $a^{k+1} - b^{k+1} = a^k \cdot a - b^k \cdot b$

$$= [b^k + m(a - b)] a - b^k \cdot b [\because a^k = bm + k(a - b)]$$

$$= b^k \cdot a + am(a - b) - b^k \cdot b$$

$$= b^k \cdot a - b^k \cdot b + am(a - b)$$

$$= b^k(a - b) + am(a - b)$$

$$= (a - b)(b^k + am) \text{ is divisible by } (a - b)$$

$\therefore P(k + 1)$ is true.

By the principle of Mathematical induction. $P(n)$ is true for all $n \in \mathbb{N}$.

$\therefore a^n - b^n$ is divisible by $a - b$ for $n \in \mathbb{N}$.

b) Structural Induction

- Deals with **recursive definition** that has
 1. **Basis**: Where one or more elementary structures are defined
 2. **Induction**: Where more complex structures are defined in terms of previously defined structures.

Example 1.19: Here is the recursive definition of a tree:

BASIS: A single node is a tree, and that node is the *root* of the tree.

INDUCTION: If $T_1, T_2, \dots, T_k$ are trees, then we can form a new tree as follows:

1. Begin with a new node N , which is the root of the tree.
2. Add copies of all the trees $T_1, T_2, \dots, T_k$.
3. Add edges from node N to the roots of each of the trees $T_1, T_2, \dots, T_k$.

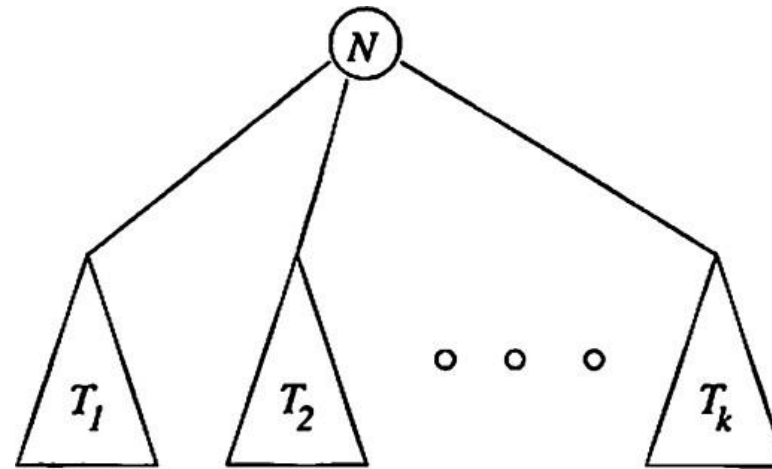


Figure 1.7: Inductive construction of a tree

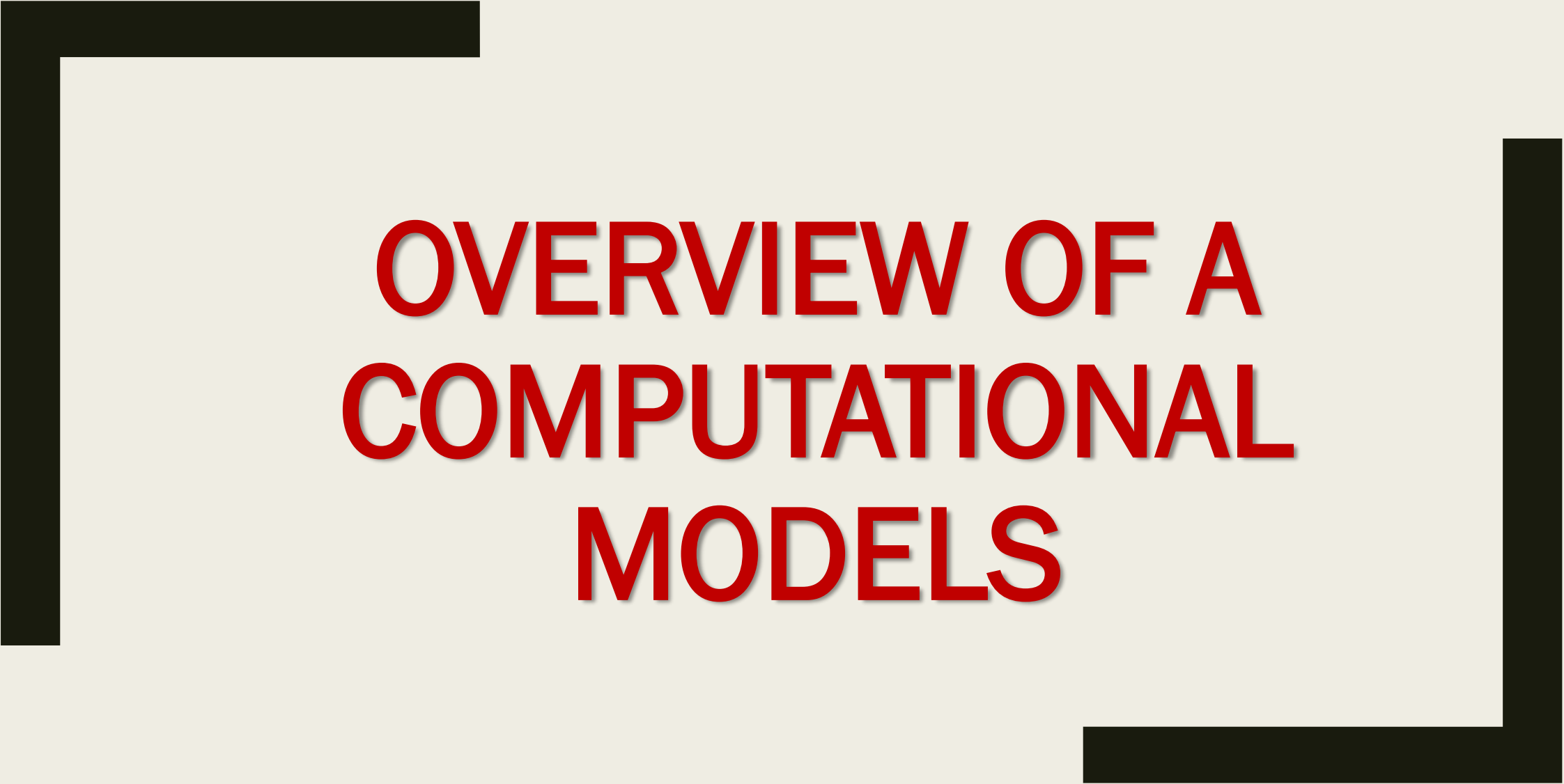
BASIS Any number or letter ie., a variable is an expression

INDUCTION If E and F are expressions, then so are $E + F$, $E * F$, and (E) .

For example,

- both 2 and x are expressions by the basis
- The inductive step tells us $x+2$, $(x+2)$ and $2+(x*2)$ are all expressions

Notice how each of these expressions depends on the previous ones being expressions

The image features two thick black L-shaped bars. One is located in the top-left corner, and the other is in the bottom-right corner. They are positioned such that they appear to frame the central text.

OVERVIEW OF A COMPUTATIONAL MODELS

Definition of Computational Models

- A computational model specifies a mechanism for computation, describing how input data is processed and transformed into output through a set of rules or operations. These models are used to:
 - Represent algorithms and their execution.
 - Analyze computational problems and their solvability.
 - Study computational complexity.

Classification of Computational Models

- Finite Automata
- Pushdown Automata
- Turing Machines

Finite Automata

- **Definition:** A Finite Automaton is a **mathematical model of computation** that consists of a finite number of states and is used to recognize patterns or decide whether a given input string belongs to a particular language.
- It reads an input string symbol by symbol and changes states according to a transition function. If the machine ends in an **accepting state**, the input is **accepted**; otherwise, it is **rejected**.

Structure of a DFA:

- States, input symbols, transition function, start state, accepting states.
- **Types:**
 - *Deterministic (DFA): Single possible next state.*
 - *Non-deterministic (NFA): Multiple possible next states.*
- **Applications:** Text search, protocol validation.
- **Importance:** Used in lexical analysis for programming languages

Pushdown Automata

- **Definition:** A Pushdown Automaton (PDA) is a type of automaton that extends **finite automata with a stack**, enabling it to recognize context-free languages (like balanced parentheses), which finite automata alone cannot handle.
- **Capabilities:** Models context-free languages.
- **Components:**
 - Non-terminal symbols, terminal symbols, production rules, start symbol.
- **Applications:**
 - *Compiler parsing.*
 - *Context-free grammar validation. **Example:** Verifying nested parentheses in code.*

Turing Machines

- **Definition:** A universal model of computation.

A Turing machine is a fundamental mathematical model of computation that manipulates symbols on an infinite tape according to a set of rules, serving as a basis for understanding algorithms and computation.

- **Components:**
 - *Infinite tape.*
 - *Read/write head.*
 - *State machine.*
- **Applications:** Foundational to complexity theory and problem decidability.

Example: Palindrome checker on a Turing Machine.

Real-World Applications of Computational Models

- **Compiler Design:** Automata and grammars for parsing.
- **Cryptography:** Turing Machines for secure computations.
- **AI and Machine Learning:** Neural networks for decision-making.
- **Algorithm Design:** Frameworks for developing efficient algorithms.

BCSE304L

Theory of Computation

Dr. ISLABUDEEN M

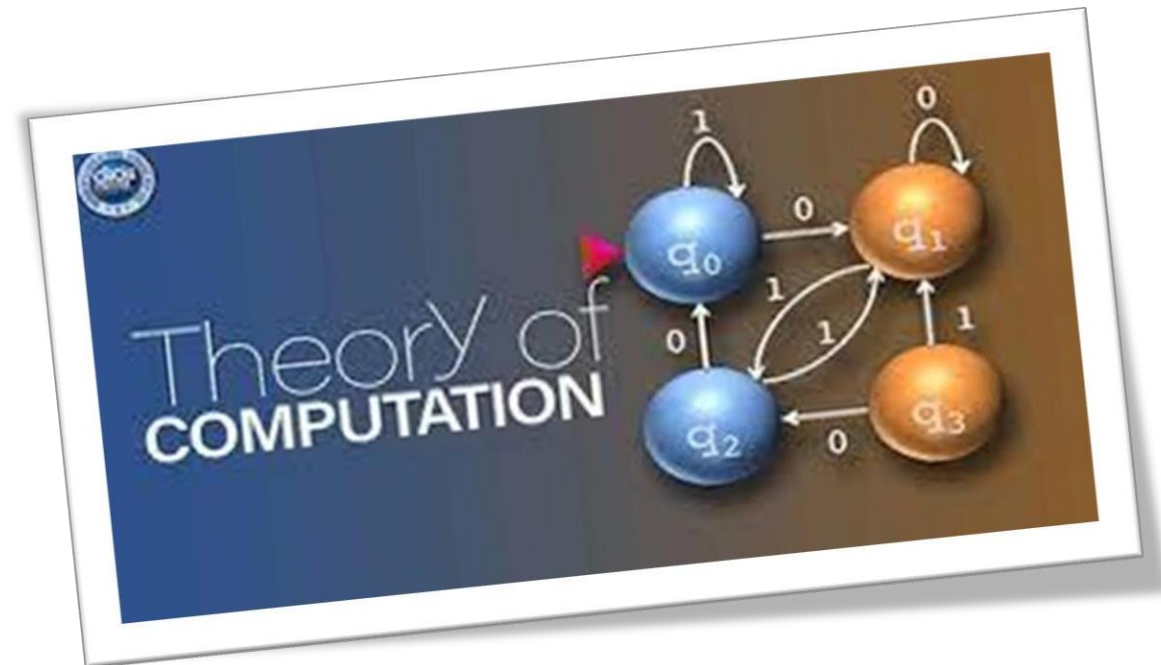
Assistant Professor Senior Grade 2

School of Computer Science and Engineering,
Vellore Institute of Technology,
Vellore, Tamil Nadu, India.

Phone No: +91 9994595561

Mail ID: islabudeen.m@vit.ac.in

Location: PRP 315 AB 23



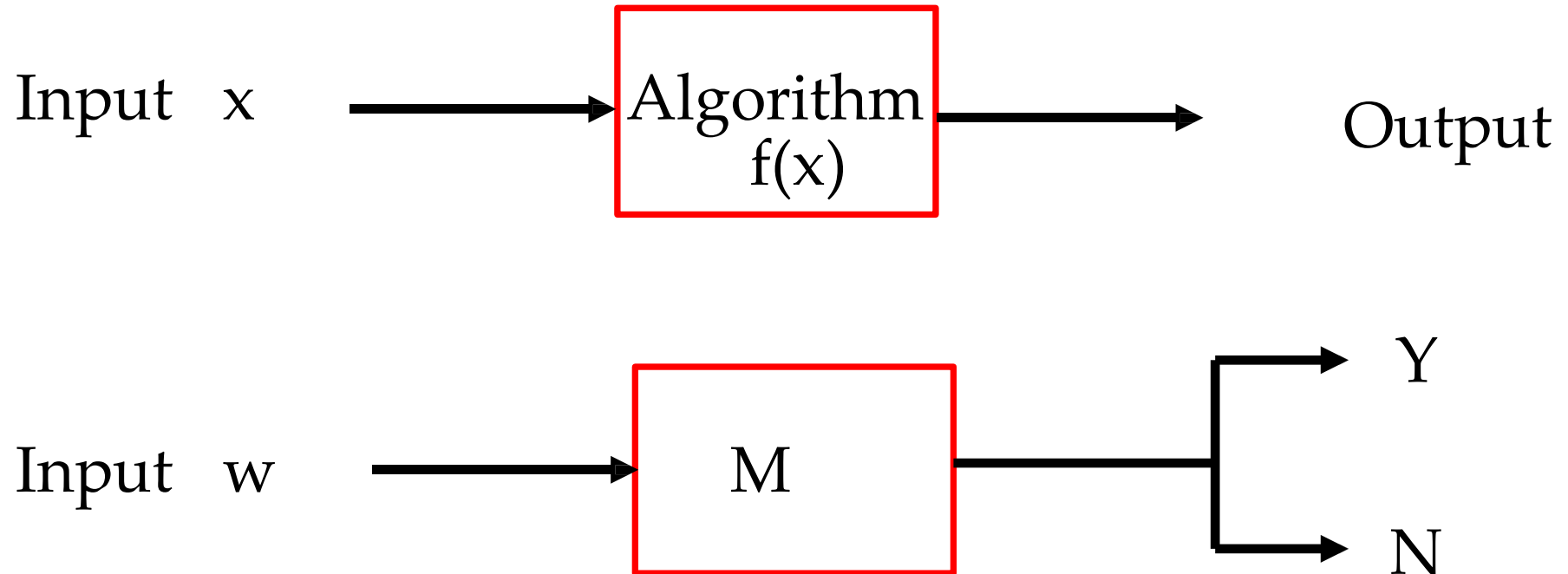
Module: 1

Introduction to Languages and Grammars

Overview of a computational model

- Computation

Program running in computer



Central Concepts of Automata Theory

- Alphabets (Σ)

- A finite, non-empty set of symbols

- Ex:

- $\alpha)$ $\Sigma = \{0, 1\}$, the binary alphabet

- $\beta)$ $\Sigma = \{a, b, \dots, z\}$, the set of all lower-case letter

- c) The set of all ASCII characters

Central Concepts of Automata Theory – cont..

• Strings

- A string (or word) is a finite sequence of symbols chosen from some alphabet

• Example

- 1011 is a string from alphabet $\Sigma = \{0, 1\}$
- Empty string, $\varepsilon \rightarrow$ a string with zero occurrences of symbols
- Length $|w|$ of string $w \rightarrow$ the number of positions for symbols in w
 - Examples: $|0111| = 4$, $|\varepsilon| = 0$, ...

Central Concepts of Automata Theory – cont..

- **Power** of an alphabet, Σ^k
 - A set of all strings of length k
- Examples
 - Given $\Sigma = \{0, 1\}$, we have
$$\Sigma^0 = \{\epsilon\}, \Sigma^2 = \{00, 01, 10, 11\}$$
 - Set of all strings over $\Sigma \rightarrow$ denoted as Σ^*
$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$
 - Set of nonempty strings from $\Sigma \rightarrow \Sigma^+$
$$\Sigma^+ = \Sigma^* - \{\epsilon\}$$
$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$
$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$
- **Concatenation** of two strings x and y is xy
 - Example
 - If $x = 01101$, $y = 110$,
 - then $xy = 01101110$, and $xx = x^2 = 0110101101, \dots$
 - ϵ is the **identity** for concatenation

since $\epsilon w = w\epsilon = w$

Central Concepts of Automata Theory – cont..

- Languages

- A language is a set of strings all chosen from some Σ^*
- If Σ is an alphabet, and $L \subseteq \Sigma^*$, then L is a language over Σ .

- Examples

- The set of all legal English words is a language $\rightarrow \{a, b, c, \dots, z\}$
- The set of all strings of n 0's followed by n 1's for $n \geq 0$ $\rightarrow \{\epsilon, 01, 0011, 000111, \dots\}$
- The set of strings of equal no. of 0's and 1's $\rightarrow \{\epsilon, 01, 10, 0011, 0101, \dots\}$
- Σ^* is an infinite language for any alphabet Σ
- ϕ = the empty language (no string or not the empty string, ϵ) is a language over any alphabet
- $\phi \neq \{\epsilon\}$

Languages – *cont...*

- Σ^* denotes the set of all finite strings over Σ
- A formal language L over the alphabet Σ is a subset of Σ^*
- That is L is a subset of Σ^*
 - Let $\Sigma = \{0, 1\}$ be the alphabet.
 - $L = \{w \in \Sigma^* \mid \text{the number of 1's in } w \text{ is even}\}$.
 - $\epsilon, 0, 00, 11, 000, 110, 101, 011, 0000, 1100, 1010, 1001, 0110, 0101, 0011, \dots$ are in L

Properties of languages

1. Concatenation
2. Substring
3. Reversal
4. Kleene Closure and Positive Closure

Concatenation of languages

- The concatenation of two languages L_1 and L_2 is given by,

$$L = L_1 L_2 = \{w \in \Sigma^* ; w = xy \text{ for some } x \in L_1 \text{ and } y \in L_2\}$$

- Concatenation of two strings w_1 and w_2 is defined as the sequence of symbols in w_1 followed by sequence of symbols in w_2
- Let $w_1 = abcd$ and $w_2 = fgh$

Then

$$w_1 w_2 = abcd fgh$$

$$w_2 w_1 = fgh abcd$$

The concatenation of string x for n times,
where $n \geq 0$, is denoted by x^n

$$x^0 = \varepsilon$$

$$x^1 = x$$

$$x^2 = x x$$

$$x^3 = x x x$$

...

Substring

Let x and y be strings over an alphabet, Σ

The string x is a substring of y if there exist strings w and z over Σ such that $y = wxz$.

- ε is a substring of every string.
- For every string x , x is a substring of x itself.

Example

- ε , *comp* and *computation* are substrings of *computation*.

Reversal

The reversal of a string is a string with same symbols in reverse order, denoted by w^R

Example

Let $w = abcd fgh$

Then

$w^R = hgfd cba$

Kleen Star

- The Kleen star of a language L is denoted by L^* which is the set of all strings obtained by concatenating zero or more strings from L

$$L^* = \{w \in \Sigma^* ; w = w_1 \dots w_k \text{ for } k \geq 0 \text{ and } w_1, w_2, \dots, w_k \in L\}$$

Let $\Sigma = \{a, b\}$ i.e. $\Sigma^1 = \{a, b\}$

$$\Sigma^2 = \Sigma^1 \cdot \Sigma^1 = \{a, b\} \cdot \{a, b\} = \{aa, ab, ba, bb\}$$

$$\Sigma^3 = \Sigma^1 \cdot \Sigma^1 \cdot \Sigma^1 = \{a, b\} \cdot \{a, b\} \cdot \{a, b\}$$

$$= \{aa, ab, ba, bb\} \cdot \{a, b\}$$

$$= \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

... etc

$$\Sigma^0 = \{\epsilon\}$$

The **Kleene closure** of Σ , denoted by Σ^* is defined as follows:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots = \{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$$

The **Positive closure** of Σ , denoted by Σ^+ is defined as follows:

$$\Sigma^+ = \Sigma^* - \{\epsilon\}$$

Operations on Languages

- Complementation
- Union
- Intersection
- Concatenation
- Reversal
- Closure

Complementation

Let L be a language over an alphabet, Σ .

The complementation of L , denoted by $\bar{L}$ (or L'), is $\Sigma^* - L$.

Example:

Let $\Sigma = \{0, 1\}$ be the alphabet.

$L = \{\omega \in \Sigma^* \mid \text{the number of 1's in } \omega \text{ is even}\}.$

$L' = \{\omega \in \Sigma^* \mid \text{the number of 1's in } \omega \text{ is not even}\}.$

Union

Let L_1 and L_2 be languages over an alphabet Σ .

The union of L_1 and L_2 , denoted by $L_1 \cup L_2$, is

$$L_1 \cup L_2 = \{x \mid x \text{ is in } L_1 \text{ or } L_2\}.$$

Example:

$$\begin{aligned} & \{x \in \{0,1\}^* \mid x \text{ begins with } 0\} \cup \{x \in \{0,1\}^* \mid x \text{ ends with } 0\} \\ &= \{x \in \{0,1\}^* \mid x \text{ begins or ends with } 0\} \end{aligned}$$

Intersection

Let L_1 and L_2 be languages over an alphabet Σ .

The intersection of L_1 and L_2 , denoted by $L_1 \cap L_2$, is

$\{x \mid x \text{ is in } L_1 \text{ and } L_2\}$.

Example:

$$\begin{aligned} & \{x \in \{0,1\}^* \mid x \text{ begins with } 0\} \cap \{x \in \{0,1\}^* \mid x \text{ ends with } 0\} \\ &= \{x \in \{0,1\}^* \mid x \text{ begins and ends with } 0\} \end{aligned}$$

Concatenation

Let L_1 and L_2 be languages over an alphabet Σ .

The concatenation of L_1 and L_2 , denoted by $L_1 \cdot L_2$, is

$$\{w_1 \cdot w_2 \mid w_1 \text{ is in } L_1 \text{ and } w_2 \text{ is in } L_2\}.$$

Example

$$L_1 = \{x \in \{0,1\}^* \mid x \text{ begins with } 0\}$$

$$L_2 = \{x \in \{0,1\}^* \mid x \text{ ends with } 0\}$$

$$L_1 \cdot L_2 = \{x \in \{0,1\}^* \mid x \text{ begins and ends with } 0 \text{ and } length(x) \geq 2\}$$

Reversal

Let L be a language over an alphabet Σ .

The reversal of L , denoted by L^R , is $\{w^R \mid w \text{ is in } L\}$.

Example

$$L = \{x \in \{0,1\}^* \mid x \text{ begins with } 0\}$$

$$L^R = \{x \in \{0,1\}^* \mid x \text{ ends with } 0\}$$

Kleene's closure

Let L be a language over an alphabet Σ .

The Kleene's closure of L , denoted by L^* , is $\{x \mid \text{for an integer } n \geq 0, x = x_1 x_2 \dots x_n \text{ and } x_1, x_2, \dots, x_n \text{ are in } L\}$.
That is, $L^* = \bigcup_{i=0}^{\infty} L_i$

Example: Let $\Sigma = \{0,1\}$ and

$$L_e^* = \{\omega \in \Sigma^* \mid \text{the number of 1's in } \omega \text{ is even}\}$$

$$(L_e)^* = \{\omega \in \Sigma^* \mid \text{the number of 1's in } \omega \text{ is odd}\}$$

Central Concepts of Automata Theory – cont...

• Grammar

- A grammar of a language, G is defined as $G = (V, T, S, P)$

- Where,

V → Finite set of objects called Non-terminals

T → Finite set of objects called Terminals

$S \in V$ → Start Symbol

P → Finite set of Productions

Ex: $G = \{(S, T), (a, b), S, P\}$

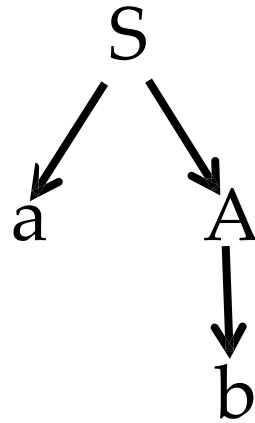
Note: $V \cap T = \phi$

Grammar (Examples)

$G = (\{S, A\}, \{a, b\}, P, S)$, where

$P: S \rightarrow aA$

$A \rightarrow b$



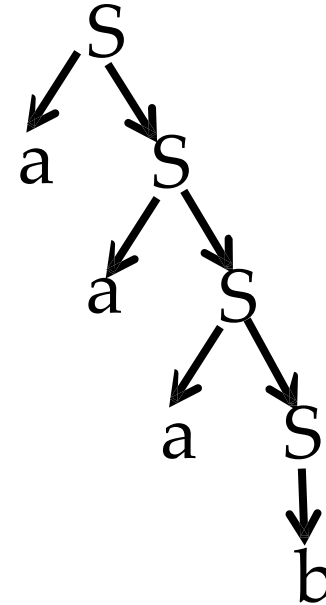
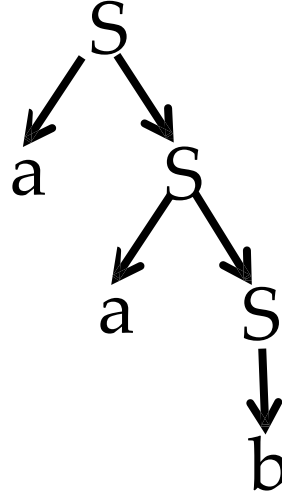
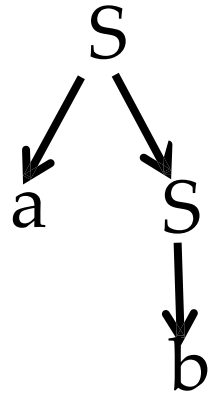
$L(G) = \{ ab \}$

Grammar

$G = (\{S\}, \{a, b\}, P, S)$, where

$P: S \rightarrow aS$

$S \rightarrow b$



$L(G) = \{ b, ab, a^2b, a^3b, \dots \}$

$L(G) = \{ a^n b \mid n \geq 0 \}$

Example problems on grammar

1. Given $\Sigma = \{a, b\}$ obtain Σ^*

- Give an example of a finite language in Σ
- Given $L = \{a^n b^n : n \geq 0\}$, check if strings $aabb$, abb are in language, L

• **Soln:**

- $\Sigma^* = \{\epsilon, a, b, ab, ba, \dots\}$
- $\{a, aa, aab\} \rightarrow$ Some example of a language in Σ

• **String:** 'aabb'

$$L = \{\epsilon, ab, aabb, aaabbb, \dots\}$$

When $n = 2$, $L = a^2 b^2$ therefore, it is a string in L

• **String:** 'abb'

This is not a string in L . Since, there is no equal number of a 's and b 's

Problem 2

- Given $u = a^2ba^3b^2$; $v = ba$. Find (i) uv (ii) uv^2 (iii) $|uv|$
- Soln:

$$(i) \quad uv = (a^2ba^3b^2).(ba) = a^2ba^3b^3a$$

$$(ii) \quad uv^2 = (a^2ba^3b^2).(ba).(ba) = a^2ba^3b^3aba$$

$$(iii) \quad |uv| = |a^2ba^3b^3a| = |aabaabbba| = 10$$

Exercise

- Consider the following two languages:

$L_1 = \{aa, bb\}$ where $\Sigma = \{a, b\}$

$L_2 = \{ppp, qqq\}$ where $\Sigma = \{p, q\}$

Compute the following:

- a) $L_3 = (L_1)^2 \cdot L_2$
- b) $L_4 = L_2 \cdot \varepsilon$
- c) $L_5 = L_2 - L_3$
- d) $L_6 = L_2 \cap L_4$
- e) $L_7 = (L_5)^*$

Solution:

$L_3 = \{aaaa, aabb, bbaa, bbbb\} \cdot \{ppp, qqq\}$

$L_4 = \{ppp, qqq\} \cdot \varepsilon = L_2$

$L_5 = L_2 = \{ppp, qqq\}$

$L_6 = \{ppp, qqq\}$

$L_7 = \{\varepsilon, ppp, qqq, pppppp, qqqqqq, \dots\}$

Problem – Construct Language from Grammar

- Given a grammar, $G = [\{S\}, \{a, b\}, S, P]$ where Production, P is given by $S \rightarrow a S b$, $S \rightarrow \epsilon$. Obtain language $L(G)$.

Soln:

$$S \Rightarrow a S b$$

$$\Rightarrow a a S b b$$

$$\Rightarrow a a a S b b b$$

$$\Rightarrow \dots$$

$$\Rightarrow a^n S b^n$$

$$\Rightarrow a^n b^n$$

$L(G) = \{a^n b^n \mid \text{where } n \geq 0\}$ or

$L(G) = \{\text{set of strings with } n \text{ a's followed by } n \text{ b's} \mid \text{where } n \geq 0\}$

Problem 1

- Given a grammar, $G = [\{A, S\}, \{a, b\}, A, P]$ where Production, P is given by $A \rightarrow a S b, S \rightarrow S b \mid \epsilon$. Obtain language $L(G)$.

Soln:

$$\begin{aligned} A &\Rightarrow a S b \\ &\Rightarrow a S b b \\ &\Rightarrow a S b b b \\ &\Rightarrow \dots \\ &\Rightarrow a S b^n \\ &\Rightarrow a b^n \end{aligned}$$

$$L(G) = \{ a b^n \mid \text{where } n \geq 1 \} \text{ or}$$

$$L(G) = \{ \text{All strings with exactly one 'a' and } n \text{ b's} \}$$

Problem 2

$G = (\{S, B\}, \{a, b, c\}, P, S)$, where

P: $S \rightarrow aSBc$ (rule 1)

$S \rightarrow abc$ (rule 2)

$cB \rightarrow Bc$ (rule 3)

$bB \rightarrow bb$ (rule 4)

$S \rightarrow abc$

$S \rightarrow aSBc \rightarrow aabcbBc \rightarrow aabBcc \rightarrow aabbcc$

$S \rightarrow aSBc \rightarrow aaSBcBc \rightarrow aaabcbBcBc \rightarrow$
 $aaabBccBc \rightarrow aaabbcbBc \rightarrow aaabbBcc \rightarrow$
 $aaabbBccc \rightarrow aabbbccc$

$$L(G) = \{ a^n b^n c^n / n \geq 1 \}$$

Problem 3

$G = (\{S\}, \{a, b, c\}, P, S)$, where

P: $S \rightarrow aSa$ (rule 1)

$S \rightarrow bSb$ (rule 2)

$S \rightarrow c$ (rule 3)

$S \rightarrow c$

$S \rightarrow aSa \rightarrow aca$

$S \rightarrow bSb \rightarrow bcb$

$S \rightarrow aSa \rightarrow abSba \rightarrow abcba$

$S \rightarrow bSb \rightarrow baSab \rightarrow bacab$

$$L(G) = \{ wcw^R / w \in \{a, b\}^* \}$$

Construction of grammar

1) $L(G) = \{ a^n c a^n \mid n \geq 0 \}$

$G = (\{S\}, \{a, c\}, P, S)$, where

$P: S \rightarrow aSa$

$S \rightarrow c$

2) $L(G) = \{ a^n b^{2n} \mid n \geq 0 \}$

$G = (\{S\}, \{a, b\}, P, S)$, where

$P: S \rightarrow aSbb \quad \text{OR} \quad P: S \rightarrow aSA, A \rightarrow bb$

$S \rightarrow \varepsilon$

$S \rightarrow \varepsilon$

3) $L(G) = \{ a^{n+2} b^n \mid n \geq 1 \}$

$G = (\{S\}, \{a, b\}, P, S)$, where

$P: S \rightarrow aSb$

$S \rightarrow aaab$

Problem 4

- Find the grammar for $\Sigma = \{0, 1\}$ that generate the set of all strings containing more 0's than 1's where 0's followed by 1's.

- Soln:

$$A \rightarrow 0 S$$

$$S \rightarrow 0 S 1 \mid \varepsilon$$

$$G = (\{A, S\}, \{0, 1\}, A, P)$$

Practice problems

$$1) L(G) = \{ a^n b^{n-3} \mid n \geq 3 \}$$

$$2) L(G) = \{ a^n b^m \mid n \geq 0, m > n \}$$

$$3) L(G) = \{ a^n b^n c^m \mid n, m \geq 1 \}$$

Solution to practice problems

1) $L(G) = \{ a^n b^{n-3} \mid n \geq 3 \}$

$G = (\{S\}, \{a, b\}, P, S)$, where

P: $S \rightarrow aSb$

$S \rightarrow aaa$

2) $L(G) = \{ a^n b^m \mid n \geq 0, m > n \}$

$G = (\{S, A\}, \{a, b\}, P, S)$, where

P: $S \rightarrow aSb \mid A$

$A \rightarrow bA \mid b$

3) $L(G) = \{ a^n b^n c^m \mid n, m \geq 1 \}$

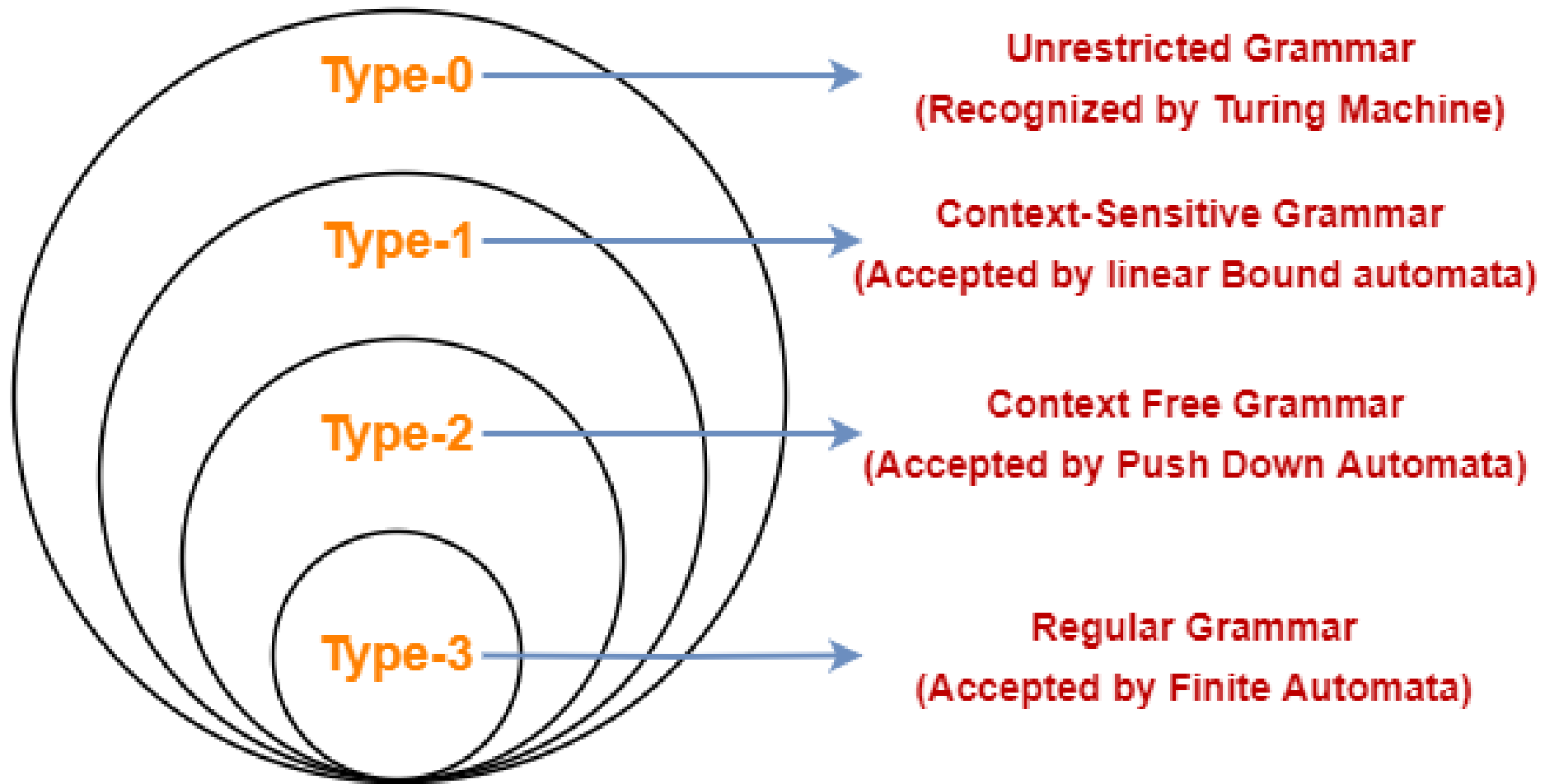
$G = (\{S, A, B\}, \{a, b, c\}, P, S)$, where

P: $S \rightarrow AB$

$A \rightarrow aAb \mid ab$

$B \rightarrow cB \mid c$

Chomsky Hierarchy – Types of grammar



Chomsky Hierarchy

	Language	Grammar	Machine
Type 3	Regular languages	Regular grammars • Right-linear grammars • Left-linear grammars	Finite-state automata
Type 2	Context-free languages	Context-free grammars	Push-down automata
Type 1	Context-sensitive languages	Context-sensitive grammars	Linear-bound automata
Type 0	Recursive languages Recursively enumerable languages	Unrestricted grammars	Turing machines

Applications

1. Traffic Lights
2. Lifts and elevators
3. Cryptography
4. Design and Analysis of Algorithm
5. Quantum Calculation
6. Logic within Computer Science
7. Randomness within Calculation



CHOMSKY HIERARCHY



- The **Chomsky Hierarchy**, formulated by Noam Chomsky in 1956, classifies grammars into four distinct types based on their generative power.
- These classifications are widely used in the field of theoretical computer science, formal language theory, and linguistics.

Type 0 Grammar (Unrestricted Grammar)

- **Definition:** The most general class of grammars, with no restrictions on production rules.
- **Form of Production Rule:** $\alpha \rightarrow \beta$, where α and β are any strings of terminal and non-terminal symbols, and $\alpha \neq \epsilon$ (empty string).
- **Key Points:**
 - *Can generate any recursively enumerable language.*
 - *Corresponds to **Turing Machines** in computational models.*
- **Example Rule:** $S \rightarrow Aba$
 $AB \rightarrow BA$

Type 1 Grammar (Context-Sensitive Grammar)

- **Definition:** Grammars where productions are length-increasing unless the derivation ends.
- **Form of Production Rule:** $\alpha A \beta \rightarrow \alpha \gamma \beta$, where A is a non-terminal and γ is a non-empty string of terminal and/or non-terminal symbols.
- **Key Points:**
 - *The length of γ must be greater than or equal to A.*
 - *Generates **context-sensitive languages**.*
 - *Corresponds to **Linear-Bounded Automata (LBA)** in computational models.*
- **Example Rule:** $A \rightarrow aB, BC \rightarrow CB$.

Type 2 Grammar (Context-Free Grammar)

- **Definition:** Each production has a single non-terminal on the left-hand side.
- **Form of Production Rule:** $A \rightarrow \gamma$, where A is a non-terminal, and γ is any string of terminal and/or non-terminal symbols.
- **Key Points:**
 - *Generates context-free languages.*
 - *Corresponds to Pushdown Automata (PDA) in computational models.*
 - *Widely used in programming languages and syntax parsing.*
- **Example Rule:** $S \rightarrow aSb$, $S \rightarrow \epsilon$.

Type 3 Grammar (Regular Grammar)

- **Definition:** The simplest grammars with strict constraints on productions.
- **Form of Production Rule:** $A \rightarrow aB$ or $A \rightarrow a$, where A and B are non-terminals, and a is a terminal.
- **Key Points:**
 - Generates *regular languages*.
 - Corresponds to *Finite Automata* in computational models.
 - Useful for defining simple patterns like regular expressions.
- **Example Rule:** $S \rightarrow aS$, $S \rightarrow a$.

Type	Name	Automaton Model	Example Language	Rule Restriction
0	Unrestricted Grammar	Turing Machine	Recursively Enumerable Languages	No restrictions on rules
1	Context-Sensitive	Linear Bounded Automata	Context-Sensitive Languages	The length of y must be greater than or equal to A
2	Context-Free	Pushdown Automaton	Context-Free Languages	One non-terminal on LHS
3	Regular Grammar	Finite Automaton	Regular Languages	RHS is aB or a